

```
In [1]: import pandas as pd
import plotly.express as px
import plotly.io as pio
pio.renderers.default = "iframe"

In [2]: pd.options.display.max_rows =200
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)
pd.set_option('display.max_colwidth', None)

In [3]: df = pd.read_parquet(r"daily-power-generation.parquet")

In [4]: df['date'] = pd.to_datetime(df['date'],format='mixed')
for col in ['region', 'state_name', 'sector', 'station_type']:
    df[col] = df[col].astype('category')

In [5]: df.info(verbose=True,show_counts=True)

<class 'pandas.DataFrame'>
RangeIndex: 2583818 entries, 0 to 2583817
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    2583818 non-null  int64
1   date                  2583818 non-null  datetime64[us]
2   region                2583818 non-null  category
3   state_name            2583818 non-null  category
4   state_code            2571173 non-null  float64
5   sector                2571173 non-null  category
6   station_type          2571173 non-null  category
7   power_station         2583818 non-null  str
8   power_station_unit    2583818 non-null  str
9   monitored_capacity    2568328 non-null  float64
10  todays_gen_prgm       2583815 non-null  float64
11  todays_gen_act        2583815 non-null  float64
dtypes: category(4), datetime64[us](1), float64(4), int64(1), str(2)
memory usage: 219.0 MB

In [6]: df.columns

Out[6]: Index(['id', 'date', 'region', 'state_name', 'state_code', 'sector', 'station_type', 'power_station', 'power_station_unit', 'monitored_capacity', 'todays_gen_prgm', 'todays_gen_act'], dtype
='str')

In [7]: df.describe()

Out[7]:
```

	id	date	state_code	monitored_capacity	todays_gen_prgm	todays_gen_act
count	2.583818e+06	2583818	2.571173e+06	2.568328e+06	2.583815e+06	2.583815e+06
mean	1.291908e+06	2021-06-11 11:49:23.657967	2.020276e+01	2.955328e+02	3.965803e+00	3.844455e+00
min	0.000000e+00	2017-01-09 00:00:00	1.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
25%	6.459542e+05	2019-06-24 00:00:00	9.000000e+00	1.100000e+02	4.200000e-01	0.000000e+00
50%	1.291908e+06	2021-07-05 00:00:00	2.200000e+01	2.100000e+02	2.840000e+00	2.060000e+00
75%	1.937863e+06	2023-05-08 00:00:00	2.700000e+01	5.000000e+02	5.520000e+00	5.530000e+00
max	2.583817e+06	2025-12-04 00:00:00	3.700000e+01	2.920000e+03	4.651000e+01	1.500000e+02
std	7.458842e+05	NaN	9.747332e+00	2.475515e+02	4.218252e+00	4.721735e+00

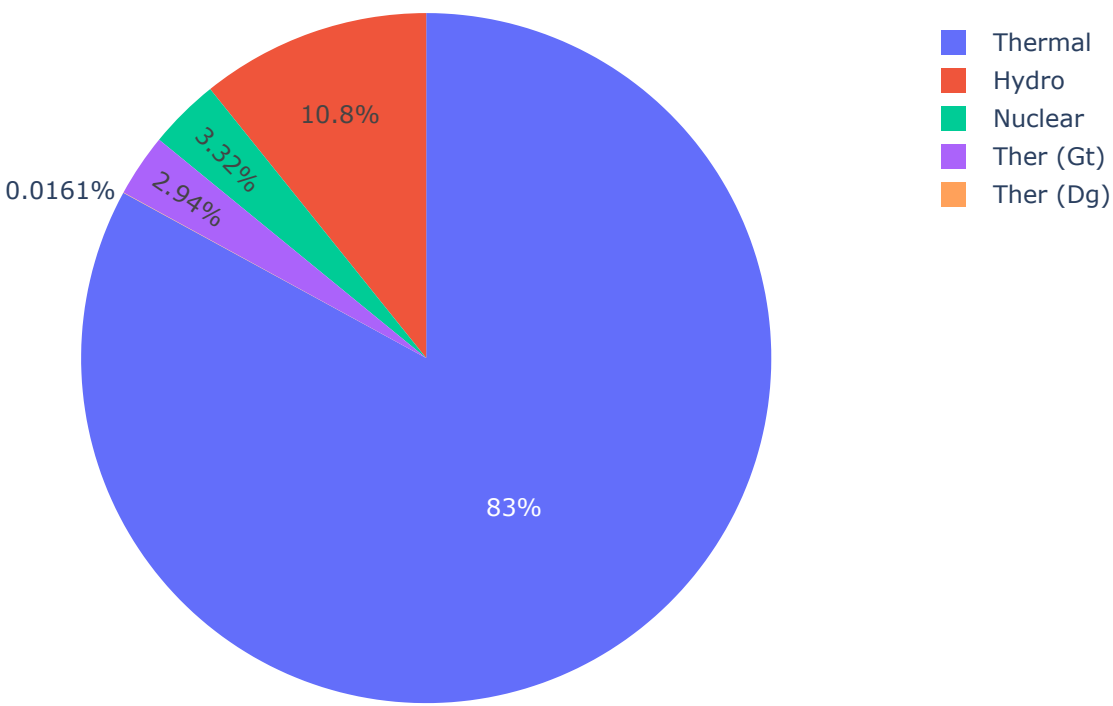
```
In [8]: df.head()

Out[8]:
```

	id	date	region	state_name	state_code	sector	station_type	power_station	power_station_unit	monitored_capacity	todays_gen_prgm	todays_gen_act
0	0	2017-01-09	Northern	Delhi	7.0	State	Thermal	Rajghat Tps	Unit 1	67.5	0.00	0.00
1	1	2017-01-09	Northern	Delhi	7.0	State	Thermal	Rajghat Tps	Unit 2	67.5	0.00	0.00
2	2	2017-01-09	Northern	Delhi	7.0	State	Ther (Gt)	I.P.Ccpp	Single Unit	270.0	1.80	0.87
3	3	2017-01-09	Northern	Delhi	7.0	State	Ther (Gt)	Pragati Ccgt-lii	Single Unit	1500.0	6.00	10.89
4	4	2017-01-09	Northern	Delhi	7.0	State	Ther (Gt)	Pragati Ccpp	Single Unit	330.4	5.49	6.53

```
In [9]: sector_pie = df.groupby('station_type')['todays_gen_act'].sum().reset_index()
sector_pie_fig=px.pie(data_frame=sector_pie,names='station_type',values='todays_gen_act',title='total electricty generation by station type',
width=700)
sector_pie_fig.show()
```

total electricty generation by station type



Grid Deficit & Reliability Analysis (Programmed vs. Actual)

Generation Variance = todays_gen_act — todays_gen_prgm

```
In [10]: df['variance'] = df['todays_gen_act'] - df['todays_gen_prgm']
```

group by region

```
In [11]: regional_perf = df.groupby('region').agg(total_programmed=('todays_gen_prgm', 'sum'),total_actual=('todays_gen_act', 'sum'), avg_daily_variance=('variance', 'mean')).reset_index()

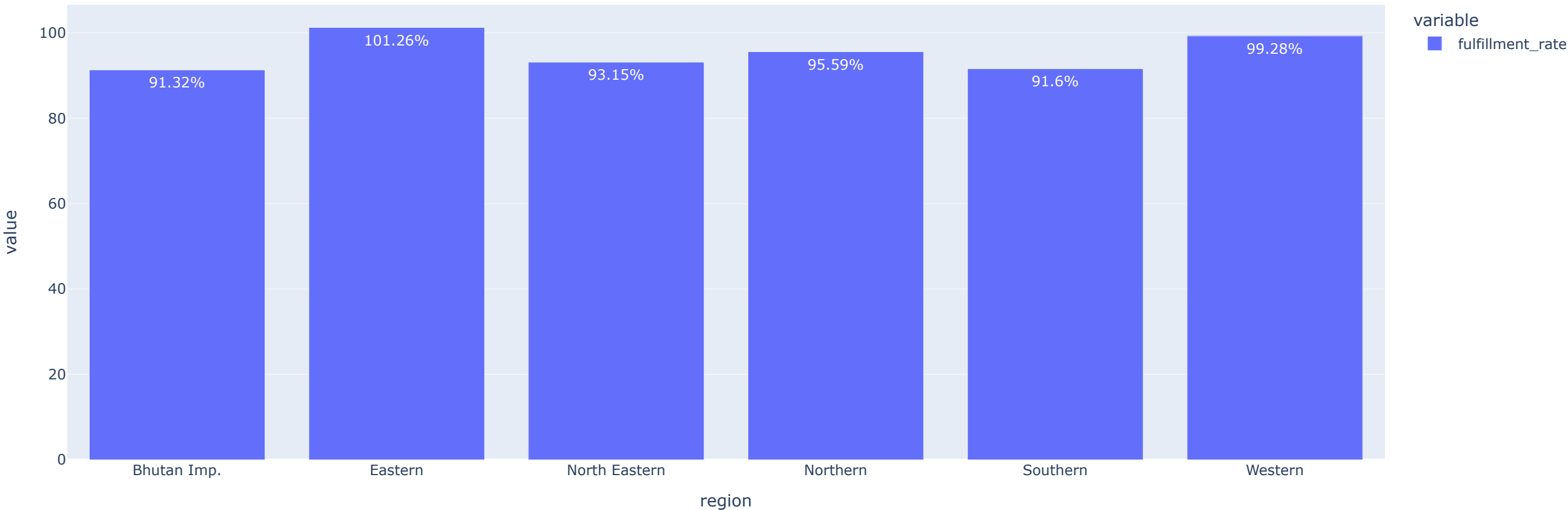
regional_perf['fulfillment_rate'] = (regional_perf['total_actual'] / regional_perf['total_programmed']) * 100
```

```
In [12]: regional_perf
```

Out[12]:

	region	total_programmed	total_actual	avg_daily_variance	fulfillment_rate
0	Bhutan Imp.	100058.10	91375.72	-0.686626	91.322662
1	Eastern	1654601.35	1675490.86	0.051247	101.262510
2	North Eastern	152521.38	142067.96	-0.142779	93.146259
3	Northern	2544073.46	2431845.16	-0.169049	95.588638
4	Southern	2099430.01	1923144.50	-0.281007	91.603173
5	Western	3696216.33	3669435.98	-0.033513	99.275466

```
In [13]: labels = regional_perf['fulfillment_rate'].round(2).astype(str) + '%'
regional_perf_fig = px.bar(data_frame=regional_perf,x='region',y=['fulfillment_rate'],text=labels,)
regional_perf_fig.show()
```



plant capacity utilization Rate (by station type)

```
In [14]: active_plants = df[df['monitored_capacity'] > 0].copy()

active_plants['max_daily_gen_mu'] = (active_plants['monitored_capacity'] * 24) / 1000

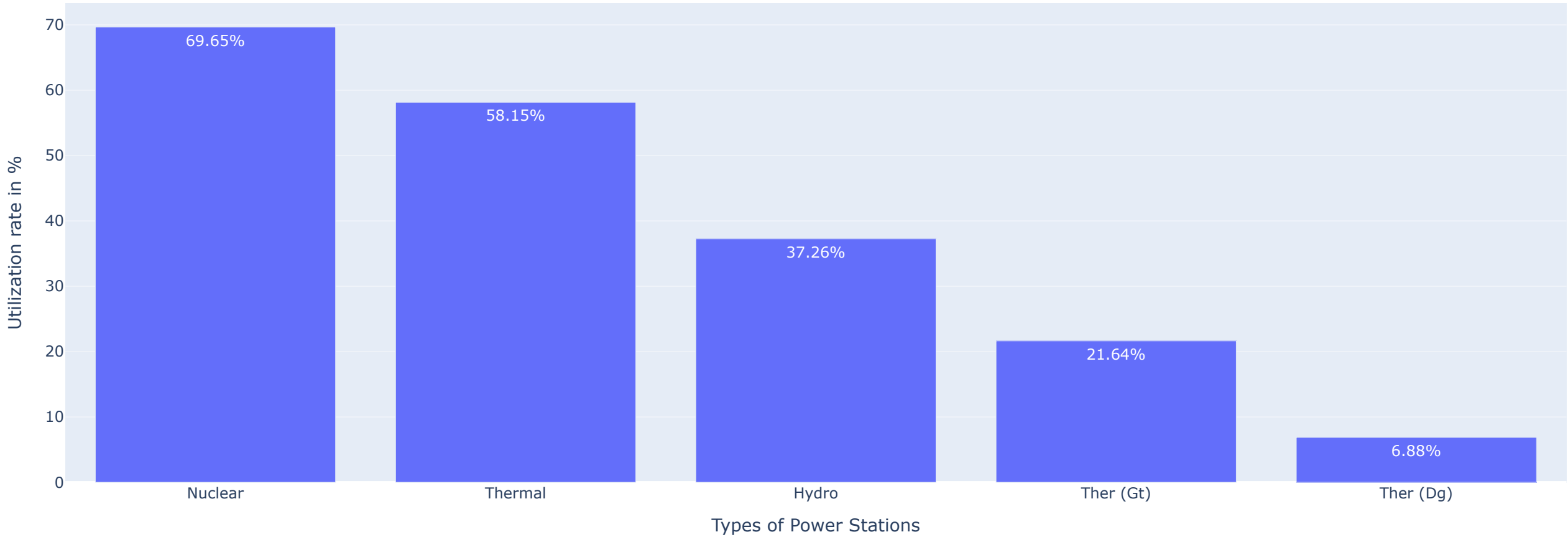
active_plants['utilization_rate_pct'] = (active_plants['todays_gen_act'] / active_plants['max_daily_gen_mu']) * 100

station_summary = active_plants.groupby('station_type')['utilization_rate_pct'].mean().sort_values(ascending=False).reset_index()
station_summary
```

Out[14]:

	station_type	utilization_rate_pct
0	Nuclear	69.654593
1	Thermal	58.152481
2	Hydro	37.264683
3	Ther (Gt)	21.643014
4	Ther (Dg)	6.875866

```
In [15]: labels = station_summary['utilization_rate_pct'].round(2).astype(str)+ '%'
station_summary_fig=px.bar(data_frame=station_summary,x='station_type',y='utilization_rate_pct',text=labels)
station_summary_fig.update_layout(xaxis_title='Types of Power Stations',yaxis_title='Utilization rate in %')
station_summary_fig.show()
```



```
In [16]: outage_threshold = 0.5
outages = df[(df['todays_gen_prgm'] > outage_threshold) & (df['todays_gen_act'] == 0)]
```

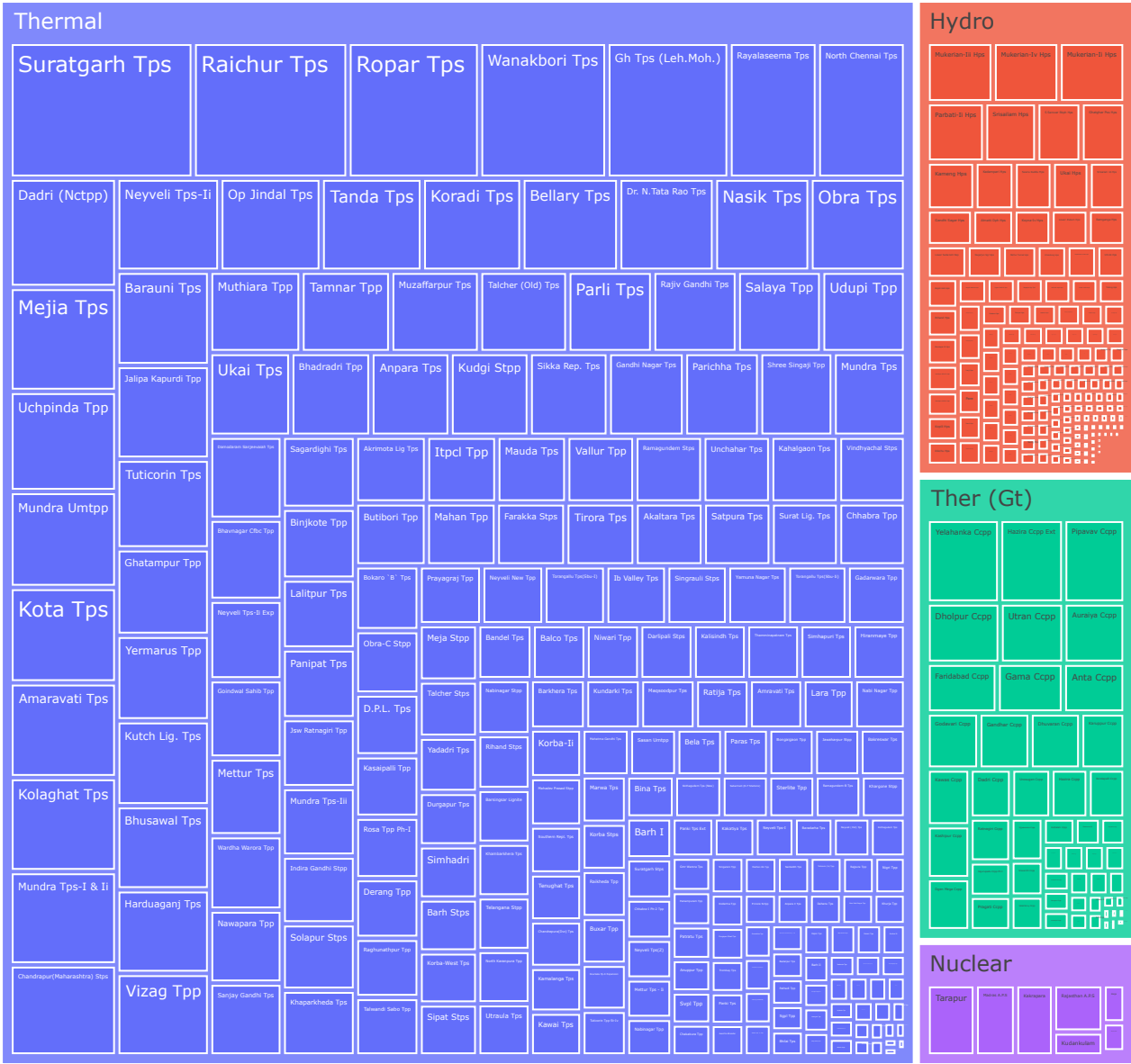
```
frequent_failures = outages.groupby(['region', 'state_name', 'state_code', 'sector','station_type', 'power_station']).size().sort_values(ascending=False).reset_index(name='outages')

In [17]: print('power stations with frequent outages :\n',frequent_failures.head(10))

power stations with frequent outages :
   region  state_name  state_code  sector  station_type  power_station  outages
0  Northern    Rajasthan      8.0   State    Thermal    Suratgarh Tps    8484
1  Southern    Karnataka     29.0   State    Thermal    Raichur Tps    7166
2  Northern    Punjab       3.0   State    Thermal    Ropar Tps    6093
3  Western     Gujarat      24.0   State    Thermal    Wanakbori Tps    5893
4  Northern    Punjab       3.0   State    Thermal    Gh Tps (Leh.Moh.)    5657
5  Southern    Andhra Pradesh  28.0   State    Thermal    Rayalaseema Tps    4088
6  Southern    Tamil Nadu    33.0   State    Thermal    North Chennai Tps    4082
7  Northern    Uttar Pradesh   9.0   Central  Thermal    Dadri (Nctpp)    3976
8  Eastern     West Bengal    19.0   Central  Thermal    Mejia Tps    3783
9  Western     Chhatisgarh    22.0   Private  Thermal    Uchpinda Tpp    3642

In [18]: frequent_failures_fig=px.treemap(data_frame=frequent_failures,path=['station_type','power_station'],values = 'outages',height=800,width=800,
title='Power Station Outages')
frequent_failures_fig.show()
```

Power Station Outages



```
In [19]: timeline = df.groupby(['date', 'station_type'])['todays_gen_act'].sum().reset_index()

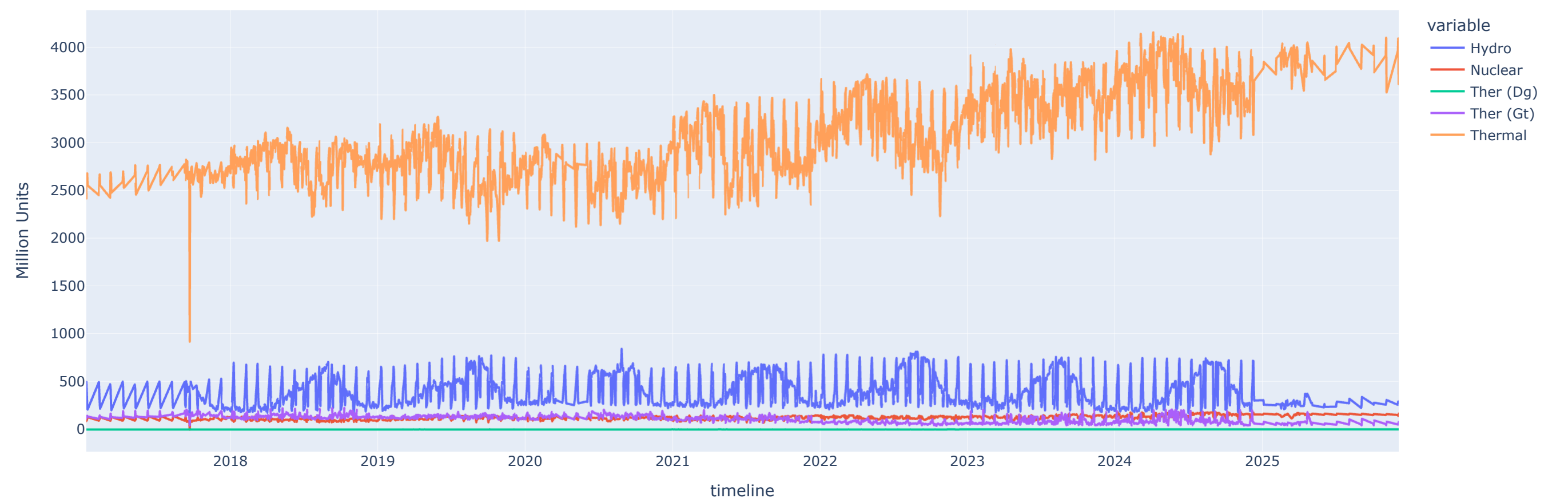
timeline_pivot = timeline.pivot(index='date', columns='station_type', values='todays_gen_act').reset_index()

timeline_pivot['Total']= timeline_pivot['Hydro'] + timeline_pivot['Nuclear'] +timeline_pivot['Ther (Dg)'] +timeline_pivot['Ther (Gt)'] + timeline_pivot['Thermal']

In [20]: timeline_pivot.head()

Out[20]: station_type      date  Hydro  Nuclear  Ther (Dg)  Ther (Gt)  Thermal    Total
0  2017-01-09    497.68    89.18    0.27    115.16    2416.96    3119.25
1  2017-01-11    262.25   109.18    0.27    143.27    2680.09    3195.06
2  2017-01-12    208.90   130.45    0.27    136.01    2554.50    3030.13
3  2017-02-09    495.63    92.28    0.27    110.56    2452.54    3151.28
4  2017-02-11    284.08   109.20    0.27    146.33    2668.72    3208.60

In [21]: timeline_pivot_fig = px.line(data_frame=timeline_pivot,x='date',y=['Hydro','Nuclear','Ther (Dg)','Ther (Gt)','Thermal'])
timeline_pivot_fig.update_layout(yaxis_title='Million Units',xaxis_title='timeline')
timeline_pivot_fig.show()
```



```
In [22]: total_fig = px.line(data_frame=timeline_pivot,x='date',y='Total',title='Total Energy generated Timeline')
total_fig.update_layout(yaxis_title='Million Units',xaxis_title='timeline')
total_fig.show()
```

